Git 真实工作完整知识体系

从零开始，所有概念先讲"是什么"、"为什么需要"，再讲"怎么用"。
聚焦真实工作场景，不只是背命令。

第一部分：Git 是什么——本质理解

1.1 一句话概括

Git 是一个**分布式版本控制系统**，用来管理代码的历史版本，支持多人协作开发。

1.2 为什么需要版本控制

没有版本控制时：

```
项目/
  ├── 代码 v1.zip
  ├── 代码 v2.zip
  ├── 代码 v2_改 bug.zip
  ├── 代码 v2_改 bug_最终版.zip
  ├── 代码 v2_改 bug_最终版_真的最终版.zip
  └── 代码 v3_重写版.zip
```

问题：谁改了什么？什么时候改的？为什么改？出问题了怎么回退？多人同时改同一个文件怎么办？

有了 Git 后：

```
git log 一条命令看到所有历史：
  abc1234 feat: 新增用户登录功能        张三   2026-09-29
  def5678 fix: 修复密码校验 bug          李四   2026-09-28
  ghi9012 feat: 新增短信验证码登录       张三   2026-09-27
```

每次修改都有记录：谁改的、什么时候改的、改了什么、为什么改。随时可以回退到任何一个版本。

1.3 Git 的核心概念

三个区域（必须搞懂）

```
工作区（Working Directory）
  │  你正在编辑的文件
  │  git add
  ▼
暂存区（Staging Area / Index）
  │  准备提交的文件快照
  │  git commit
  ▼
本地仓库（Local Repository）
  │  提交的历史记录
  │  git push
  ▼
远程仓库（Remote Repository，如 GitHub/GitLab）
```

- **工作区**：你电脑上的文件，你正在编辑的地方
- **暂存区**：一个"准备区"，你通过 `git add` 把要提交的文件放到这里
- **本地仓库**：`git commit` 后，暂存区的内容变成一条提交记录，存在本地
- **远程仓库**：`git push` 后，本地提交同步到远程（GitHub/GitLab），团队其他人可以拉取

类比：

- 工作区 = 你的办公桌（正在干活的地方）
- 暂存区 = 快递打包台（把要寄的东西放上去）
- 本地仓库 = 你家仓库（打包好的快递存在这里）
- 远程仓库 = 快递中转站（发到那里，别人也能收到）

分支（Branch）

- 是什么：代码的一条独立开发线
- 为什么需要：多人开发时，每个人在自己的分支上工作，互不干扰。开发完了合并到主分支
- 类比：写论文时，你在"草稿版"上改，不影响"正式版"。改好了再合并到"正式版"

```
main（主分支，随时可以部署的版本）
  │
  ├── feature/login（张三开发登录功能）
```

```
  ├── feature/payment（李四开发支付功能）
  └── bugfix/fix-crash（王五修复崩溃 bug）
```

第二部分：日常开发流程——真实工作中怎么用 Git

2.1 标准开发流程（最常用）

这是你在公司每天重复的流程：

第 1 步：拉取最新代码

```
git pull origin main
```

第 2 步：创建功能分支

```
git checkout -b feature/xxx
```

第 3 步：写代码...

第 4 步：查看改动

```
git status          ← 看哪些文件改了
git diff            ← 看具体改了什么内容
```

第 5 步：暂存并提交

```
git add .           ← 把所有改动放到暂存区
git commit -m "feat: 新增 xxx 功能"
```

第 6 步：推送到远程

```
git push origin feature/xxx
```

第 7 步：提 Pull Request / Merge Request

（在 GitLab/GitHub 网页上操作，请求合并到 main）

第 8 步：同事 Code Review，通过后合并

2.2 每天的实际操作

早上到公司

```
# 切换到主分支，拉取最新代码
git checkout main
git pull origin main

# 切回自己的功能分支，合并最新代码
git checkout feature/xxx
git merge main
# 如果有冲突 → 解决冲突 → git add . → git commit
```

为什么要先拉 main 再合并：因为别人可能已经往 main 合并了新代码，你需要把最新代码合到自己的分支，提前解决冲突，不要等到提 PR 时才发现冲突一堆。

开发中

```
# 随时查看状态
git status

# 查看具体改动
git diff                    # 工作区 vs 暂存区
git diff --cached           # 暂存区 vs 本地仓库
git diff main               # 当前分支 vs main 分支

# 提交
git add 具体文件.java        # 只暂存某个文件（推荐，不要无脑 git add .）
git commit -m "feat: 描述"
```

```
# 发现刚才提交写错了，修改最后一次提交
git commit --amend          # 修改提交信息或补充文件
```

下班前

```
# 推送到远程
git push origin feature/xxx

# 如果别人也推了，可能有冲突
git pull origin feature/xxx  # 先拉最新的
# 解决冲突 → git add . → git commit → git push
```

2.3 提交信息规范（Conventional Commits）

公司一般要求提交信息有固定格式：

```
<type>(<scope>): <subject>

feat(user): 新增手机验证码登录功能
fix(order): 修复订单金额计算精度丢失问题
docs(readme): 更新部署文档
style(format): 代码格式化，无逻辑变更
refactor(cache): 重构缓存逻辑，提取公共方法
test(user): 新增用户注册单元测试
chore(deps): 升级 Spring Boot 版本到 2.7.18
```

type	含义	使用场景
feat	新功能	新增了一个功能
fix	修复 bug	修了一个 bug
docs	文档	只改了文档
style	格式	空格、分号等，不影响逻辑
refactor	重构	代码结构变了，功能不变
test	测试	新增或修改测试
chore	杂项	构建、依赖升级等
perf	性能	性能优化
ci	CI/CD	改了 CI 配置

为什么要有规范：

- 一眼看出这次提交改了什么类型
- 可以自动生成 CHANGELOG
- `git log --oneline` 时清晰可读

第三部分：分支管理——团队协作的核心

3.1 常见分支模型

模型一：Feature Branch（小团队最常用）

```
main ──────────────────●──────────────→
                       ↑
  feature/A ──┘        feature/B ──┘
  （开发完合并回 main）   （开发完合并回 main）
```

- 只有 main 一个长期分支
- 每个人从 main 拉功能分支，开发完合并回去
- 适合 3-5 人小团队

模型二：Git Flow（经典模型，中大型团队）

```
main（生产环境代码）
  ↑ 合并
release/1.0（发布分支，测试修复）
  ↑ 合并
develop（开发主分支）
  ↑ 合并
feature/login  feature/payment  bugfix/xxx
（功能分支）     （功能分支）       （修复分支）
```

分支类型：

分支	从哪来	合并到哪	用途	命名
`main`	—	—	生产环境代码，随时可部署	`main`
`develop`	main	main	开发主分支，集成功能	`develop`
`feature/*`	develop	develop	新功能开发	`feature/login`
`bugfix/*`	develop	develop	修复 bug	`bugfix/fix-crash`
`release/*`	develop	main + develop	发布前测试	`release/1.0`
`hotfix/*`	main	main + develop	紧急修复线上问题	`hotfix/fix-payment`

模型三：GitHub Flow（最简单，互联网常用）

```
main ──────────────────────────●──────────→
          ↑                    ↑
  feature/A ──PR 合并──→   feature/B ──PR 合并──→
```

- 只有 main 一个长期分支
- 功能分支开发完提 PR，Review 通过后直接合并到 main
- 合并后自动部署
- GitHub、很多互联网公司用这个

3.2 合并 vs 变基（merge vs rebase）

这是面试高频问题，也是实际工作中必须理解的。

merge（合并）

```
main:     A ── B ──────────── D
                ↑            ↑
feature:    C ──┘────────────┘
                       merge commit
```

结果：`A ── B ── C ── D（merge commit）`
保留了完整的分支历史，但历史线会分叉

rebase（变基）

```
main:     A ── B ──────────── D
                ↑            ↑
feature:    C ──┘            |
                       rebase 后
feature:          C' ────────┘
```

结果：`A ── B ── C' ── D`
历史线是直的，更干净，但改写了历史

区别总结：

维度	merge	rebase
历史形状	分叉，有 merge commit	直线，干净
是否改写历史	否	是（C 变成了 C'）
安全性	安全，保留完整历史	有风险，改写后不能推给共享分支
适用场景	合并功能分支到 main	同步 main 到自己的功能分支

实际工作中的用法：

```
# 同步 main 到自己的功能分支 → 用 rebase（保持历史干净）
git checkout feature/xxx
git rebase main

# 功能分支开发完合并到 main → 用 merge（保留合并记录）
git checkout main
git merge feature/xxx
# 或者用 --no-ff 强制生成 merge commit（即使能快进也生成）
git merge --no-ff feature/xxx
```

黄金规则：不要在已经推送到远程的公共分支上 rebase。rebase 会改写提交历史，如果别人已经基于你的提交开发了，rebase 后别人的提交会出问题。

第四部分：冲突解决——工作中最常遇到的问题

4.1 冲突是什么

两个人改了同一个文件的同一行，Git 不知道该保留谁的，就报冲突。

```
<<<<<<< HEAD（你的代码）
    return "你好，世界";
=======
    return "Hello, World";
>>>>>>> feature/xxx（别人的代码）
```

4.2 解决冲突的步骤

```
# 1. 拉取代码时出现冲突
git pull origin main
# CONFLICT (content): Merge conflict in xxx.java

# 2. 打开冲突文件，手动编辑
# 删除 <<<<<<<、=======、>>>>>>> 标记
# 保留正确的代码（可能是你的、别人的、或者两者的结合）

# 3. 标记冲突已解决
git add xxx.java

# 4. 完成合并
git commit
# 或者如果是 rebase
git rebase --continue
```

4.3 避免冲突的技巧

1. **频繁拉取 main 代码**：每天至少 pull 一次，不要攒一周再合并
2. **小步提交**：不要一次改 100 个文件再提交，改一点提交一点
3. **沟通**：如果两个人要改同一个文件，提前沟通谁先改
4. **代码模块化**：每个人负责不同的模块，减少改同一个文件的概率

第五部分：撤销操作——改错了怎么办

5.1 撤销的三种场景

```
工作区 ←── git checkout / git restore ←── 暂存区 ←── git reset ←── 本地仓库
                                                                    ↑
                                                                git push
                                                                    ↑
                                                                远程仓库
```

场景一：工作区改错了，还没 add

```
# 撤销某个文件的修改（恢复到最近一次 commit 或 add 的状态）
git restore xxx.java

# 撤销所有文件的修改
git restore .
```

场景二：已经 add 了，还没 commit

```
# 从暂存区撤回到工作区
git restore --staged xxx.java

# 或者用老写法
git reset HEAD xxx.java
```

场景三：已经 commit 了，还没 push

```
# 方式 1：撤销最后一次提交，保留改动在工作区
git reset --soft HEAD~1
# 改动还在，可以重新 commit

# 方式 2：撤销最后一次提交，改动也撤回到工作区
```

```
git reset --mixed HEAD~1
# 默认行为，等同于 git reset HEAD~1

# 方式 3：撤销最后一次提交，改动也丢弃（危险！）
git reset --hard HEAD~1
# 改动全部丢失，无法恢复

# 方式 4：只修改提交信息
git commit --amend
```

场景四：已经 push 了

```
# 用 revert 创建一个"反向提交"，抵消之前的改动
git revert <commit_hash>
# 这会生成一个新的提交，内容是撤销指定提交的改动
# 安全！不会改写历史，适合公共分支

# 如果一定要强制覆盖远程（危险！）
git push --force-with-lease origin feature/xxx
# --force-with-lease 比 --force 安全，会检查远程有没有别人的新提交
```

5.2 三种 reset 的区别（面试高频）

模式	工作区	暂存区	本地仓库	用途
`--soft`	不变	不变	回退	撤销 commit，保留所有改动在暂存区，可以重新 commit
`--mixed`（默认）	不变	回退	回退	撤销 commit 和 add，改动回到工作区
`--hard`	回退	回退	回退	全部撤销，改动丢失

第六部分：实用技巧——提升效率

6.1 暂存（Stash）——临时保存现场

场景：你在 feature 分支上改了一半，突然要切到 main 修 bug，但又不想提交半成品。

```
# 保存当前改动（工作区和暂存区都保存）
git stash save "正在开发登录功能"

# 切到别的分支修 bug...
git checkout main
# 修 bug...提交...

# 切回来，恢复之前的改动
git checkout feature/xxx
git stash pop
# pop = 恢复并删除 stash
# 如果只想恢复不删除：git stash apply
```

6.2 查看历史

```
# 查看提交历史（简洁版）
git log --oneline --graph --all
# 效果：
# * abc1234 (HEAD -> feature/xxx) feat: 新增登录
# | * def5678 (main) fix: 修复 bug
# |/
# * ghi9012 feat: 初始化项目

# 查看某个文件的修改历史
git log --oneline -- xxx.java

# 查看某次提交改了什么
git show abc1234

# 查看某个文件在某次提交中的改动
```

```
git show abc1234 -- xxx.java

# 搜索提交信息
git log --grep="登录"

# 查看谁改了什么（blame）
git blame xxx.java
# 每一行前面显示：commit hash 作者 时间 内容
```

6.3 cherry-pick（挑选提交）

场景：feature/A 分支上有一个 commit 修了一个紧急 bug，你想把这个修复单独拿到 main 分支，但不想合并整个 feature/A 分支。

```
# 在 main 分支上
git checkout main
git cherry-pick abc1234
# 把 abc1234 这个提交"复制"到当前分支
```

6.4 标签（Tag）

场景：发布 v1.0 版本时打个标签，方便以后回退。

```
# 打标签
git tag v1.0.0
git tag v1.0.0 -m "发布 1.0 版本"

# 推送标签到远程
git push origin v1.0.0

# 查看所有标签
git tag

# 切换到某个标签
git checkout v1.0.0
```

6.5 .gitignore

是什么：告诉 Git 哪些文件不要追踪。

```
# 编译产物
target/
build/
dist/

# IDE 配置
.idea/
.vscode/
*.iml

# 依赖
node_modules/

# 敏感文件
.env
*.key

# 日志
*.log

# 系统文件
.DS_Store
Thumbs.db
```

注意：如果文件已经被 Git 追踪了，再加到 .gitignore 是无效的。需要先 `git rm --cached` 取消追踪。

```
git rm --cached .env
# 然后加到 .gitignore
```

第七部分：团队协作规范——真实公司的做法

7.1 Code Review 流程

```
开发者提 PR/MR
    ↓
自动触发 CI（编译、测试、代码检查）
    ↓
CI 通过 → 指定 Reviewer（至少 1-2 人）
    ↓
Reviewer 看代码，提意见
    ↓
开发者修改 → 重新 push → Reviewer 再看不通过 → Approve
    ↓
合并到 main
```

Review 关注什么：

- 逻辑是否正确
- 有没有潜在的 bug
- 代码风格是否符合规范
- 有没有安全风险（SQL 注入、XSS 等）
- 性能有没有问题

7.2 分支保护规则（Branch Protection）

公司一般会对 main 分支设置保护：

- **禁止直接 push**：不能直接 `git push origin main`，必须通过 PR 合并
- **要求 Code Review**：至少 1-2 人 Approve 才能合并
- **要求 CI 通过**：自动化测试必须通过才能合并
- **禁止 force push**：不能强制推送

7.3 提交粒度

好的提交：一次提交只做一件事，提交信息清晰描述改了什么。

✓ 好的提交历史：

```
abc1234 feat(user): 新增手机验证码登录
def5678 feat(user): 新增邮箱验证码登录
ghi9012 fix(user): 修复验证码过期时间计算错误
```

✗ 不好的提交历史：

```
abc1234 开发登录功能（包含登录、注册、忘记密码、修了 3 个 bug、改了配置）
```

7.4 回滚线上代码

场景：刚部署了新版本，发现线上有严重 bug，需要立即回退。

```
# 方式 1：用 revert 撤销（安全，推荐）
git revert <有问题的 commit_hash>
git push origin main

# 方式 2：回退到上一个版本（如果刚部署就发现问题）
git reset --hard <上一个好的 commit_hash>
git push --force-with-lease origin main
# 注意：force push 会改写远程历史，要确保团队知道
```

第八部分：面试高频问题

Q1：git merge 和 git rebase 的区别？

merge 会生成一个 merge commit，保留完整的分支历史，历史线会分叉。rebase 会把当前分支的提交"搬"到目标分支的最新位置，历史线是直的更干净，但改写了提交历史。实际工作中，同步 main 到自己的功能分支用 rebase，功能分支合并到 main 用 merge。黄金规则是不要在已经推送到远程的公共分支上 rebase。

Q2：git reset 的三种模式有什么区别？

soft 模式只回退本地仓库，改动保留在暂存区，可以重新 commit。mixed 模式（默认）回退本地仓库和暂存区，改动回到工作区。hard 模式全部回退，改动丢失。实际工作中，撤销 commit 用 soft 或 mixed，hard 要慎用因为改动会丢失。

Q3：线上代码出问题了怎么回滚？

用 git revert 创建一个反向提交来抵消有问题的提交，这是最安全的方式，不会改写历史。如果刚部署就发现问题，也可以用 git reset 回退到上一个好的 commit 然后 force push，但这会改写远程历史，需要团队配合。

Q4：你平时的工作流是怎样的？

每天早上先切到 main 分支 pull 最新代码，然后切回自己的功能分支 rebase main 同步最新代码。开发过程中小步提交，每次提交只做一件事，提交信息遵循 Conventional Commits 规范。开发完后推送到远程，提 PR 请求合并到 main。同事 Code Review 通过后合并。合并前确保 CI 测试全部通过。

Q5：遇到冲突怎么解决？

先 git pull 拉取最新代码，Git 会标记冲突文件。打开文件找到冲突标记（<<<<<<<、=======、>>>>>>>），和同事沟通确认保留哪部分代码，手动编辑解决冲突。然后 git add 标记冲突已解决，git commit 完成合并。避免冲突的技巧是频繁拉取 main 代码、小步提交、不同人负责不同模块。

Q6：git stash 的使用场景？

当你在功能分支上改了一半代码，突然需要切到别的分支修紧急 bug，但又不想提交半成品时，用 git stash 把当前改动临时保存起来。修完 bug 切回来后，用 git stash pop 恢复之前的改动继续开发。

Q7：如何保证提交历史干净？

三个做法。第一，提交信息遵循 Conventional Commits 规范，type 加 scope 加描述，一眼看出改了什么。第二，同步 main 到自己的分支时用 rebase 而不是 merge，保持历史线是直的。第三，合并功能分支到 main 时用 merge --no-ff，保留合并记录，方便追溯。

Q8：force push 和 force-with-lease 的区别？

force push 会无条件覆盖远程分支，如果别人在你之后推了新提交，那些提交会被覆盖丢失。force-with-lease 会先检查远程分支有没有别人的新提交，如果有就拒绝推送，更安全。实际工作中如果必须 force push，一定要用 force-with-lease。

附录：Git 命令速查表

日常操作

命令	作用
`git status`	查看当前状态（哪些文件改了）
`git diff`	查看工作区的具体改动
`git add <file>`	暂存文件
`git add .`	暂存所有改动
`git commit -m "msg"`	提交到本地仓库
`git commit --amend`	修改最后一次提交
`git push origin <branch>`	推送到远程
`git pull origin <branch>`	拉取并合并远程代码
`git fetch origin`	只拉取不合并

分支操作

命令	作用
`git branch`	查看本地分支
`git branch -a`	查看所有分支（含远程）
`git checkout -b <branch>`	创建并切换分支
`git checkout <branch>`	切换分支
`git branch -d <branch>`	删除本地分支
`git merge <branch>`	合并分支
`git rebase <branch>`	变基到目标分支
`git rebase --continue`	rebase 冲突解决后继续

命令	作用
`git rebase --abort`	放弃 rebase

撤销操作

命令	作用
`git restore <file>`	撤销工作区改动
`git restore --staged <file>`	从暂存区撤出
`git reset --soft HEAD~1`	撤销 commit，保留在暂存区
`git reset --mixed HEAD~1`	撤销 commit 和 add
`git reset --hard HEAD~1`	全部撤销（危险）
`git revert <commit>`	创建反向提交（安全）
`git stash save "msg"`	暂存当前改动
`git stash pop`	恢复暂存的改动
`git stash list`	查看所有暂存

查看历史

命令	作用
`git log --oneline`	简洁查看提交历史
`git log --oneline --graph --all`	图形化查看所有分支历史
`git log --grep="关键词"`	搜索提交信息
`git log -- <file>`	查看文件的修改历史
`git show <commit>`	查看某次提交的详情
`git blame <file>`	查看文件每行是谁改的
`git reflog`	查看所有操作记录（包括 reset 丢失的）

远程操作

命令	作用
`git remote -v`	查看远程仓库地址
`git remote add origin <url>`	添加远程仓库
`git push --force-with-lease`	安全地强制推送
`git fetch --prune`	拉取远程分支并清理已删除的
`git cherry-pick <commit>`	挑选某个提交到当前分支
`git tag <name>`	打标签
`git tag -d <name>`	删除标签

配置

```
# 全局配置（所有项目通用）
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
git config --global core.editor "code --wait"    # 默认编辑器
git config --global init.defaultBranch main       # 默认分支名

# 项目级配置（只影响当前项目）
git config user.name "项目专用名字"

# 查看配置
git config --list
```